Lecture # 7

By: Muhammad Zain Uddin

email: zuddin@iba.edu.pk

Good Read

HTTPS://RESEARCH.SWTCH.COM/HWMM

Agenda

Locks continue

Open MP in detail

Attribution

TODAY'S LECTURE IS BASED ON STANFORD'S COURSE ON PARALLEL COMPUTING AND HTTPS://RESEARCH.SWTCH.COM/HWMM

Litmus Test: Write Queue

Can this program see r1 = 0, r2 = 0?

```
// Thread 1        // Thread 2
x = 1              y = 1
r1 = y             r2 = x
```

On sequentially consistent hardware: no.

On x86 (or other TSO): yes!

Litmus Test: Independent Reads of Independent Writes (IRIW)

Can this program see r1 = 1, r2 = 0, r3 = 1, r4 = 0?

(Can Threads 3 and 4 see x and y change in different orders?)

```
// Thread 1    // Thread 2    // Thread 3    // Thread 4
x = 1                  y = 1          r1 = x                r3 = y
                       r2 = y         r4 = x
```

On sequentially consistent hardware: no.

On x86 (or other TSO): no.

Barrier

To fix algorithms that depend on stronger memory ordering, non-sequentially-consistent hardware supplies explicit instructions called memory barriers (or fences) that can be used to control the ordering.

We can add a memory barrier to make sure that each thread flushes its previous write to memory before starting its read

```
// Thread 1        // Thread 2
x = 1              y = 1
barrier            barrier
r1 = y             r2 = x
```

Preliminaries: some terminology

Deadlock

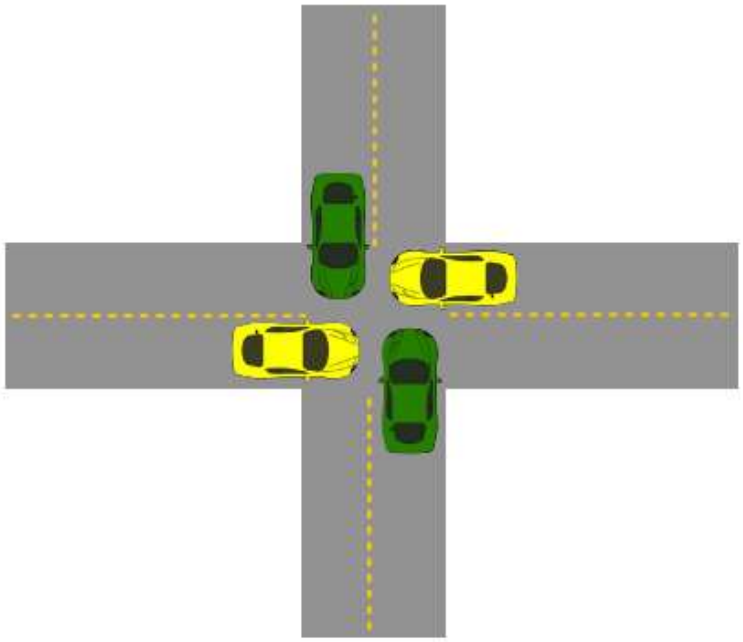
Livelock

Starvation

Deadlock and livelock concern program correctness.

Starvation is really an issue of fairness.

Deadlock



Deadlock is a state where a system has outstanding operations to complete, but no operation can make progress.

Deadlock can arise when each operation has acquired a <u>shared resource</u> that another operation needs.

In a deadlock situations, there is no way for any thread (or, in this illustration, a car) to make progress unless some thread relinquishes a resource ("backs up")

Deadlock

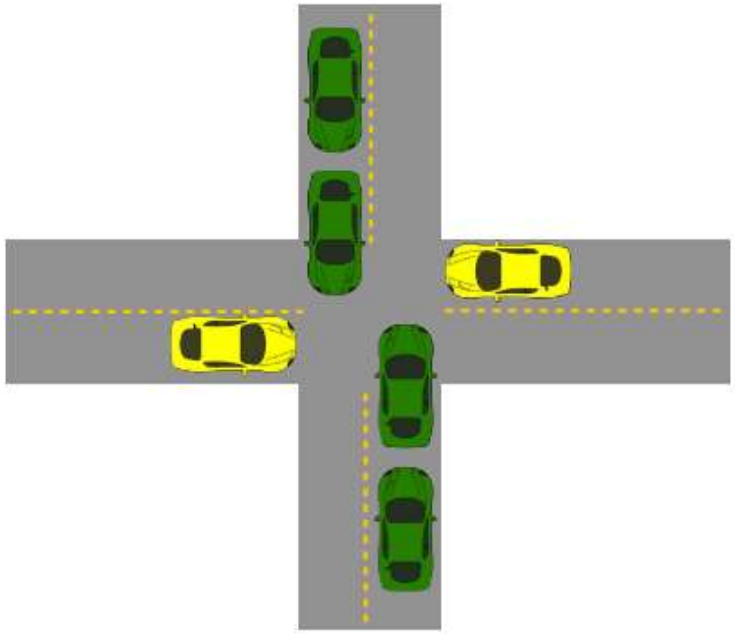
Mutual exclusion: only one processor can hold a given resource at once

Hold and wait: processor must hold the resource while waiting for other resources it needs to complete an operation

No preemption: processors don’t give up resources until operation they wish to perform is complete

Circular wait: waiting processors have mutual dependencies (a cycle exists in the resource dependency graph)

Starvation



State where a system is making overall progress, but some processes make no progress.
(green cars make progress, but yellow cars are stopped)

Starvation is usually not a permanent state
(as soon as green cars pass, yellow cars can go)

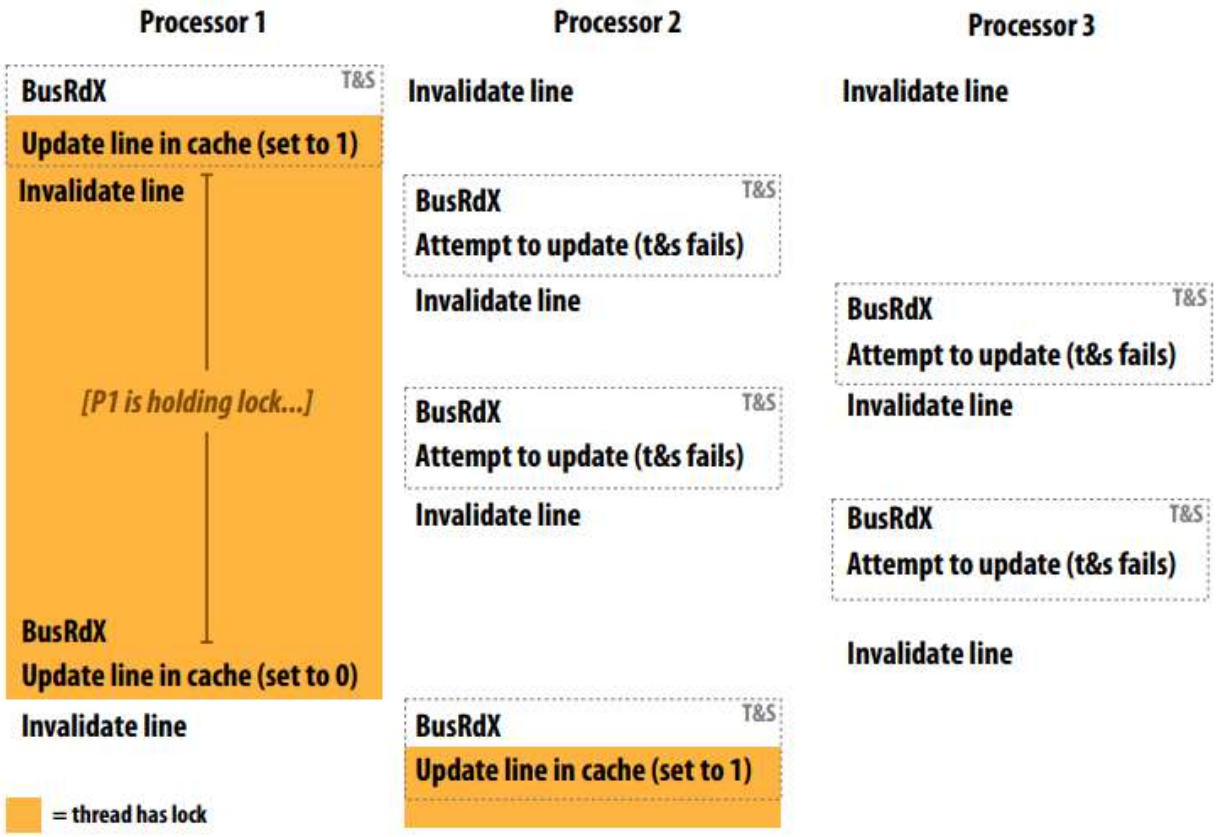
Test and Set Based Lock

Atomic test-and-set instruction:

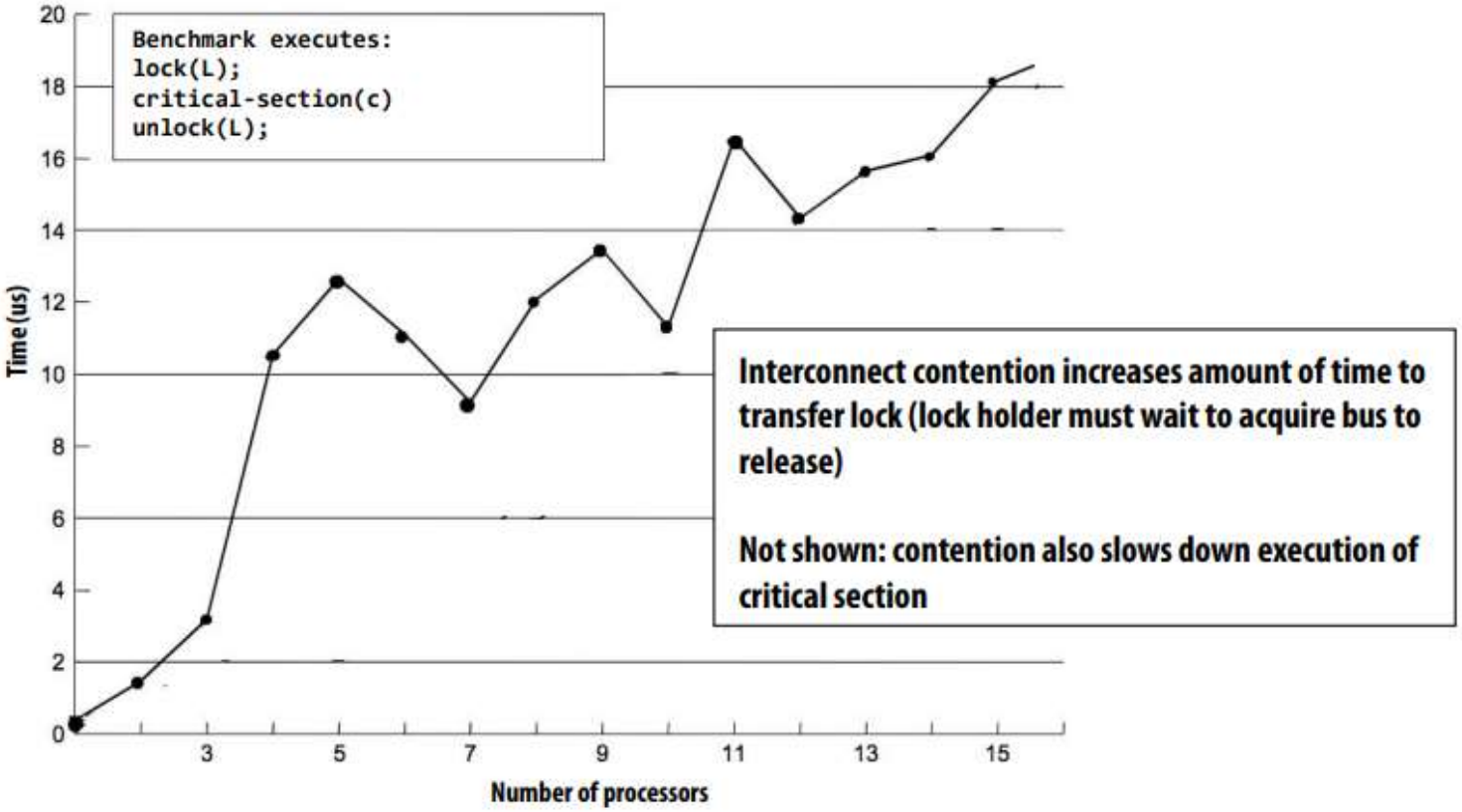
```
ts R0, mem[addr]        // load mem[addr] into R0
                        // if mem[addr] is 0, set mem[addr] to 1
```

```
lock:       ts   R0, mem[addr]        // load word into R0
            bnz  R0, lock             // if 0, lock obtained

unlock:     st   mem[addr], #0        // store 0 to address
```

Test and Set Lock: Coherence Traffic



Test and Set Lock Performance



Desirable lock characteristics

Low latency

Low interconnect traffic

Scalability

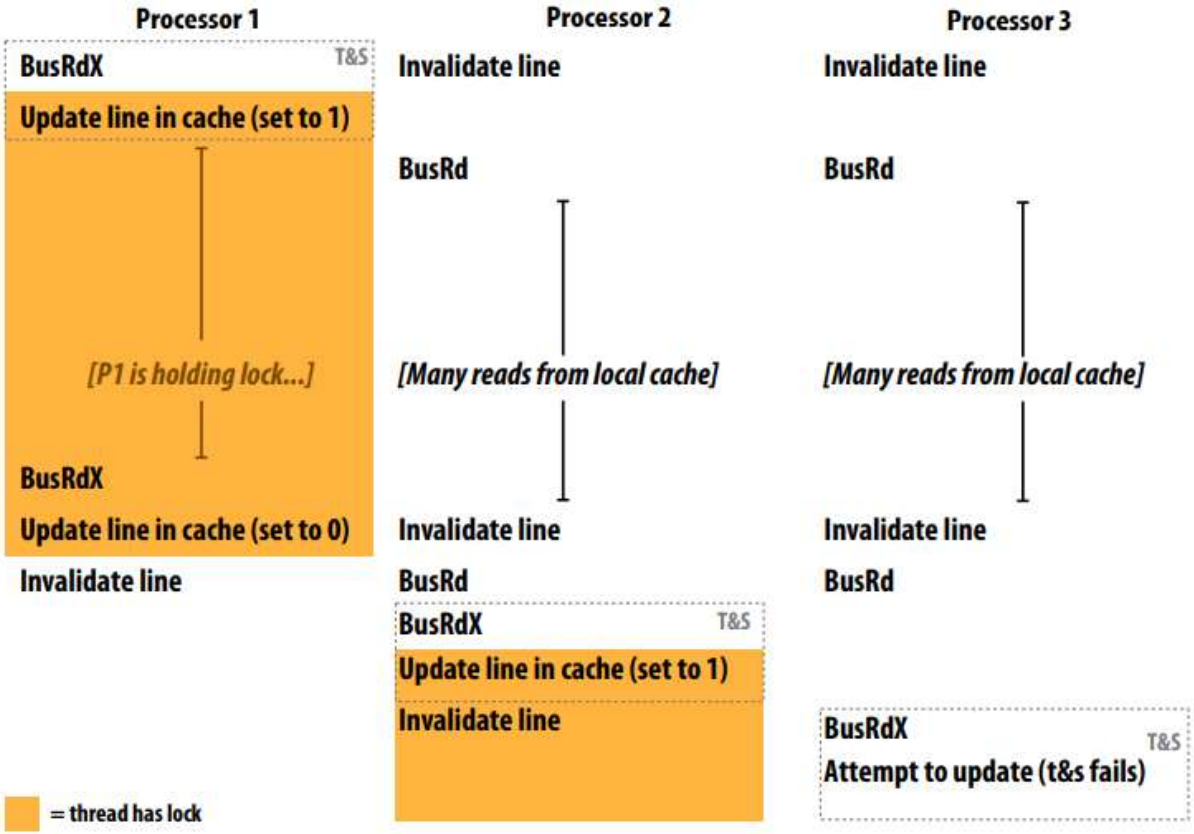
Low storage cost

Fairness

Test-and-test-and-set lock

```
void Lock(int* lock) {
        while (1) {

        while (*lock != 0); // while another processor has the lock…
                            // (assume *lock is NOT register allocated)



        if (test_and_set(*lock) == 0)     // when lock is released, try to acquire it
        return;
        }

}

void Unlock(int* lock) {

 *lock = 0;

}
```

Test and Test and Set: Coherence Traffic



Test-and-test-and-set characteristics

Slightly higher latency than test-and-set in no contention case

Generates much less interconnect traffic

More scalable

Storage cost unchanged

Still no provisions for fairness

Comparison

Lock Type	Mechanism	Performance
Test-and-Set (TAS)	Always writes (test_and_set in loop)	High contention, inefficient for many threads
Test-and-Test-and-Set (TTAS)	Reads first (if (lock == 0), then test_and_set)	Reduces cache contention, better performance

Ticket lock

```
struct lock {
   int next_ticket;
   int now_serving;
};

void Lock(lock* l) {
  int my_ticket = atomic_increment(&l->next_ticket);   // take a "ticket"
  while (my_ticket != l->now_serving);                 // wait for number to be called
}

void unlock(lock* l) {
  l->now_serving++;
}
```

No atomic operation needed to acquire the lock (only a read)

Result: only one invalidation per lock release (O(P) interconnect tra!c)

Example: Sorted Linked List

What can go wrong if multiple threads operate on the linked list simultaneously?

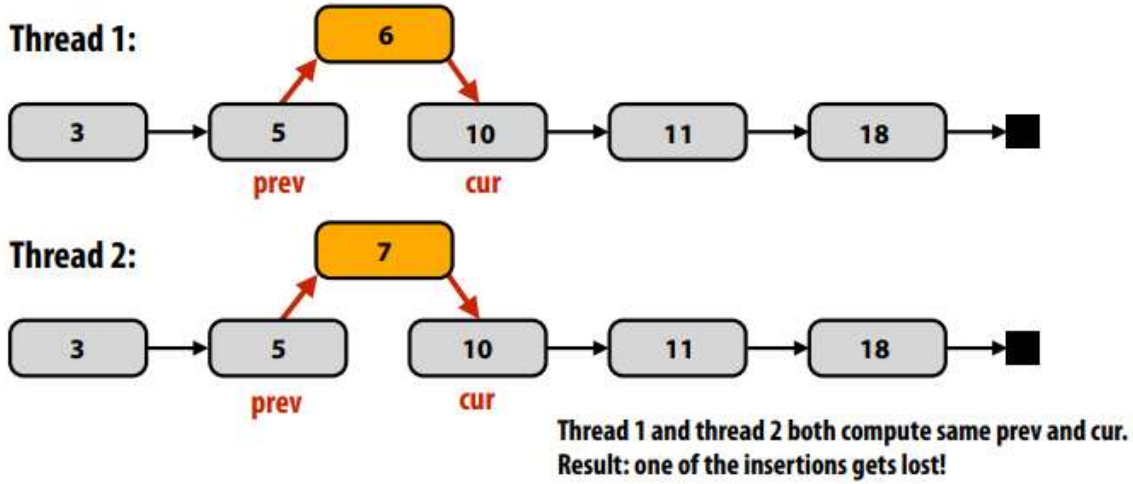
```
struct Node {          struct List {
   int value;            Node* head;
   Node* next;         };
};
```

```
void insert(List* list, int value) {

   Node* n = new Node;
   n->value = value;

   // assume case of inserting before head of
   // of list is handled here (to keep slide simple)

   Node* prev = list->head;
   Node* cur = list->head->next;

   while (cur) {
     if (cur->value > value)
       break;

     prev = cur;
     cur = cur->next;
   }

   n->next = cur;
   prev->next = n;
}
```

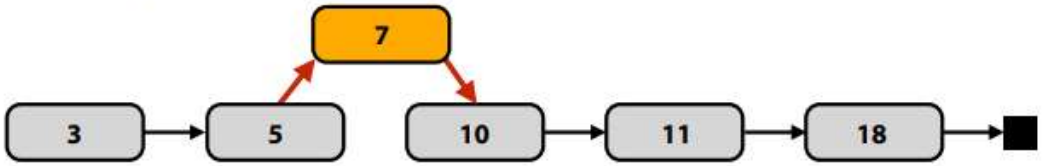
```
void delete(List* list, int value) {

   // assume case of deleting first node in list
   // is handled here (to keep slide simple)

   Node* prev = list->head;
   Node* cur = list->head->next;

   while (cur) {
     if (cur->value == value) {
       prev->next = cur->next;
       delete cur;
       return;
     }

     prev = cur;
     cur = cur->next;
   }
}
```

Example: simultaneous insertion

Thread 1 attempts to insert 6
Thread 2 attempts to insert 7

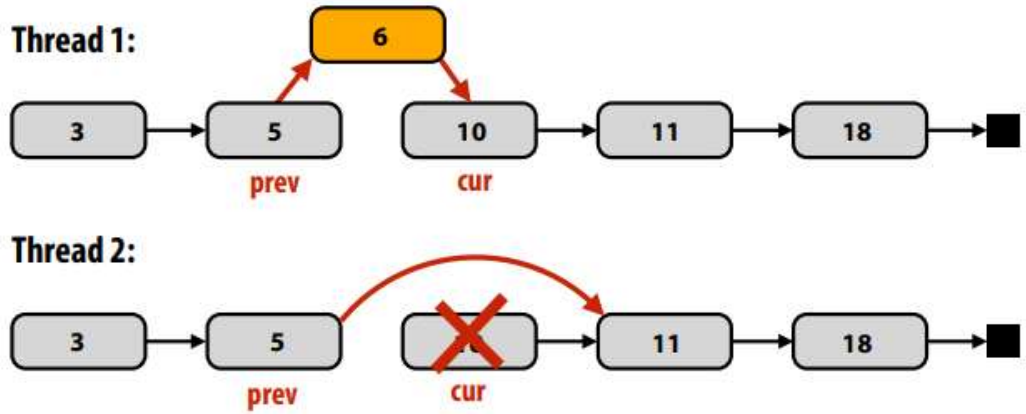


Result: (assuming thread 1 updates `prev->next` before thread 2)

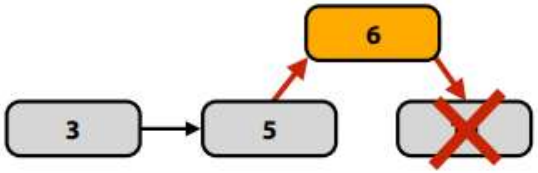


Example: simultaneous insertion/deletion

Thread 1 attempts to insert 6
Thread 2 attempts to delete 10



Possible result: (thread 2 finishes delete first)



Question

HOW CAN YOU SOLVE THE ISSUE?

Solution: Protect list with a single lock

```
struct Node {
   int value;
   Node* next;
};

struct List {
  Node* head;
  Lock  lock;
};
```

Per-list lock

```
void insert(List* list, int value) {

   Node* n = new Node;
   n->value = value;

   lock(list->lock);

   // assume case of inserting before head of
   // of list is handled here (to keep slide simple)

   Node* prev = list->head;
   Node* cur = list->head->next;

   while (cur) {
     if (cur->value > value)
       break;

     prev = cur;
     cur = cur->next;
   }
   n->next = cur;
   prev->next = n;
   unlock(list->lock);
}
```

```
void delete(List* list, int value) {

   lock(list->lock);

   // assume case of deleting first element is
   // handled here (to keep slide simple)

   Node* prev = list->head;
   Node* cur = list->head->next;

   while (cur) {
     if (cur->value == value) {
       prev->next = cur->next;
       delete cur;
       unlock(list->lock);
       return;
     }

     prev = cur;
     cur = cur->next;
   }
   unlock(list->lock);
}
```

Question

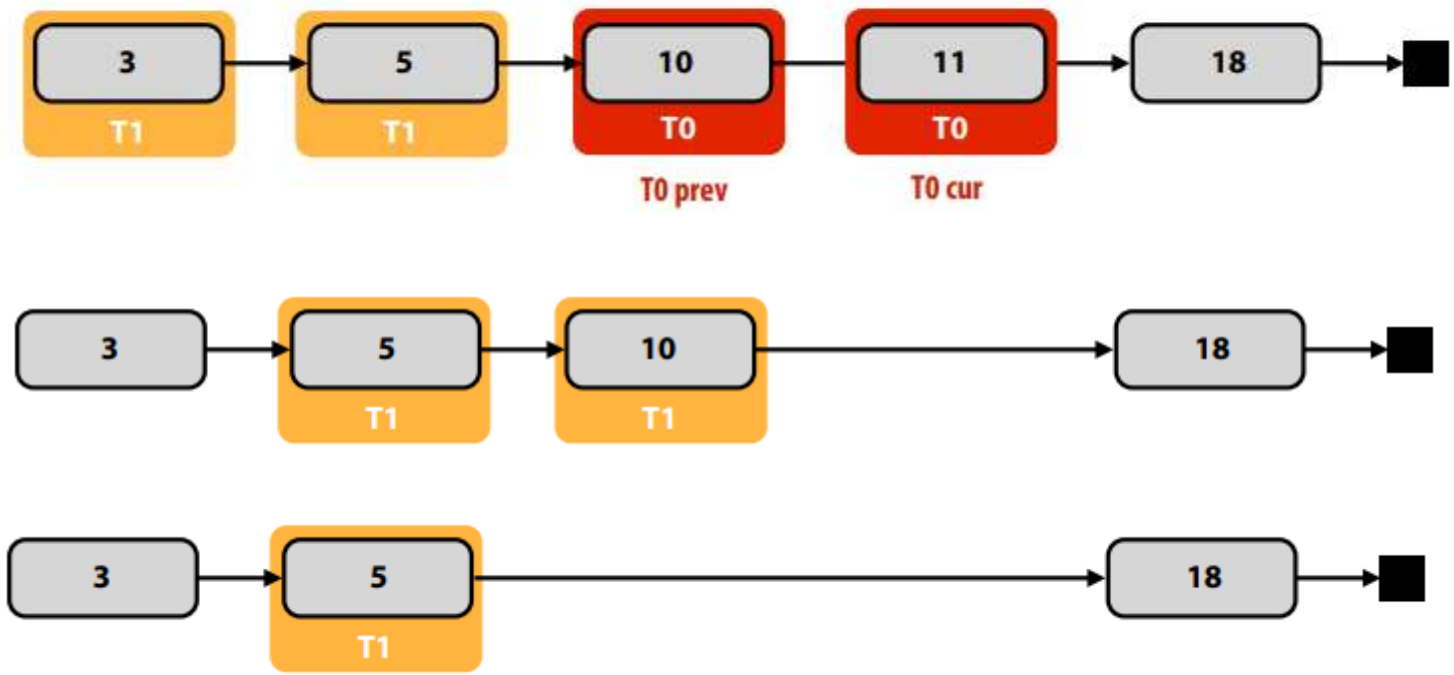
WHAT ARE THE ISSUES AND WHAT ARE THE ADVANTAGES?

Question

CAN YOU DO BETTER?

Solution 2: “hand-over-hand” locking

Thread 0: delete(11)
Thread 1: delete(10)



Blocking algorithms/data structures

A blocking algorithm allows one thread to prevent other threads from completing operations on a shared data structure indefinitely

Example:

- Thread 0 takes a lock on a node in our linked list
- Thread 0 is swapped out by the OS, or crashes, or is just really slow (takes a page fault), etc.
- Now, no other threads can complete operations on the data structure (although thread 0 is not actively making progress modifying it)

An algorithm that uses locks is blocking regardless of whether the lock implementation uses spinning or pre-emption

Lock Free Data Structures

Example: atomic_min()

```
// atomicCAS:  ("compare and swap")
// performs the following logic atomically
int atomicCAS(int* addr, int compare, int val) {
   int old = *addr;
   *addr = (old == compare) ? val : old;
   return old;
}
```

```
void atomic_min(int* addr, int x) {
   int old = *addr;
   int new = min(old, x);
   while (atomicCAS(addr, old, new) != old) {
     old = *addr;
     new = min(old, x);
   }
}
```

Why is atomic_min() a Lock-Free Approach?

A lock-free approach ensures that multiple threads can safely update a shared variable without using explicit locks (e.g., mutexes, spinlocks, or semaphores). Instead of blocking other threads while one thread updates a value, it relies on atomic operations, specifically Compare-And-Swap (CAS).

How Does It Avoid Locks?

Instead of locking the variable before modification, it tries to update it optimistically using atomicCAS().

If another thread has modified the value in the meantime, the function simply retries (while loop), without blocking other threads.

This makes it non-blocking, meaning no thread gets stuck waiting for another.

Role of Compare-And-Swap (CAS)

The atomicCAS() function works as follows:

Reads the current value at addr.

If addr still contains old, it updates it to new atomically.

Otherwise, it fails and returns the latest value, requiring the caller to retry

Why is This Lock-Free?

No thread gets blocked: Unlike locks, if a thread fails to update addr, it retries immediately.

Progress is guaranteed: At least one thread will always succeed in updating addr, ensuring forward progress.

Avoids deadlocks and priority inversion: Since no explicit locking mechanism is used, threads don’t wait indefinitely.

Approach	Mechanism	Blocking?	Fairness	Performance	Scalability	Use Case
Test-and-Set (TAS) Lock	Atomic test_and_set, spinlocks	Yes (Spins)	No	High contention, CPU overhead	Poor	Short critical sections
Test-and-Test-and-Set (TTAS) Lock	Checks before atomic set	Yes (Spins)	No	Reduces cache contention	Moderate	Slightly improved spinlock
Handoff Lock (MCS, Ticket Lock)	Queue-based handover	Yes	Yes	Reduces contention, avoids spinning	Good	Many threads, fairness required
Mutex (Blocking Lock)	OS-managed blocking	Yes (Threads sleep)	Yes	Avoids CPU waste but adds context switch overhead	Good	When fairness & low CPU usage matter
Semaphore	Counter-based, allows multiple accesses	Yes (if using wait)	Yes	Moderate overhead due to kernel interaction	Good	Managing access to a limited resource (e.g., database connections)
Lock-Free (CAS-based)	Uses atomic compare-and-swap (CAS)	No	No (depends on algorithm)	Very fast, avoids context switching	Excellent	High-performance, concurrent data structures